

Ch1: Computer networks & internet

tabla de contenido

1. [[### Internet vista con tuercas y tornillos |Internet visto con tuercas y tornillos]]
- 2.

Introducción

Internet vista con tuercas y tornillos



putacionales conectados donde **Millones de dispositivos com-

- Host: final del sistema



- Corriendo aplicaciones en los bordes del internet
Switch de paquetes : paquetes (trozos de paquetes)

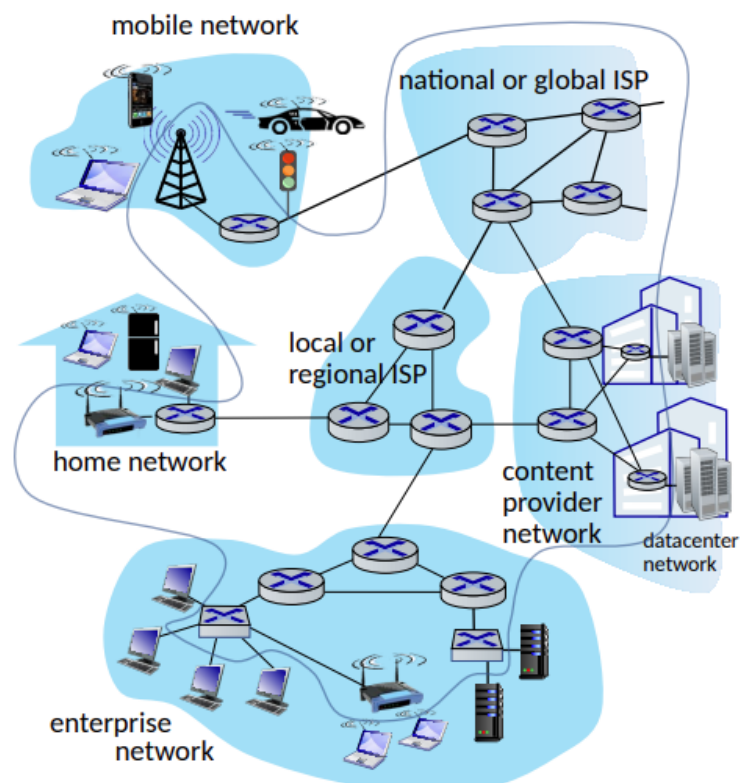


- Routers y switcher de comunicación **
- Fibra, cobre , radio, satélite, etc.

**Canales

- Tasa de transmisión: ancho de banda
- Colección de dispositivos, routes y conexiones, administrada por una orga-

Redes



nización.

El Internet es la red de redes

- Son ISPs interconectados **Protocolos esta en todos lados**
- Control de envío y recepción de mensajes
- HTTP, Streaming, Skype, etc.. **Estándares del Internet**
- RFC: solicitud de comentarios
- IETF: Ingenieros de fuerza de tarea del internet

Internet con vista a los servicios

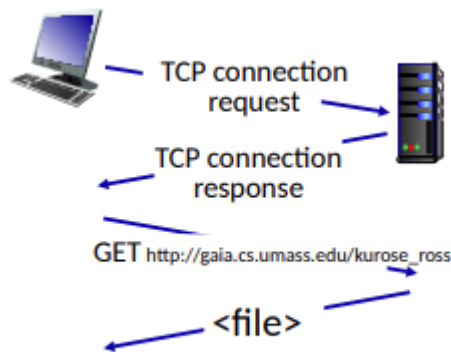
Infraestructura que provee servicios para aplicaciones

- web, Streaming, Multimedia, Correo, etc.. **Provee de una interfaz de programación para aplicaciones distribuidas**
- "Hooks", nos permite enviar y recibir aplicaciones para conectarnos y usar los medios de transporte del Internet
- Provee opciones de servicios, análogos a servicio postal

¿ Que es un protocolo?

Usado por computadores donde los protocolos manejan todo el Internet

Los protocolos definen el formato, orden de los mensajes enviados y recibidos entre entidades de redes, y acciones tomadas en transmisión de mensajes, recibirlo.

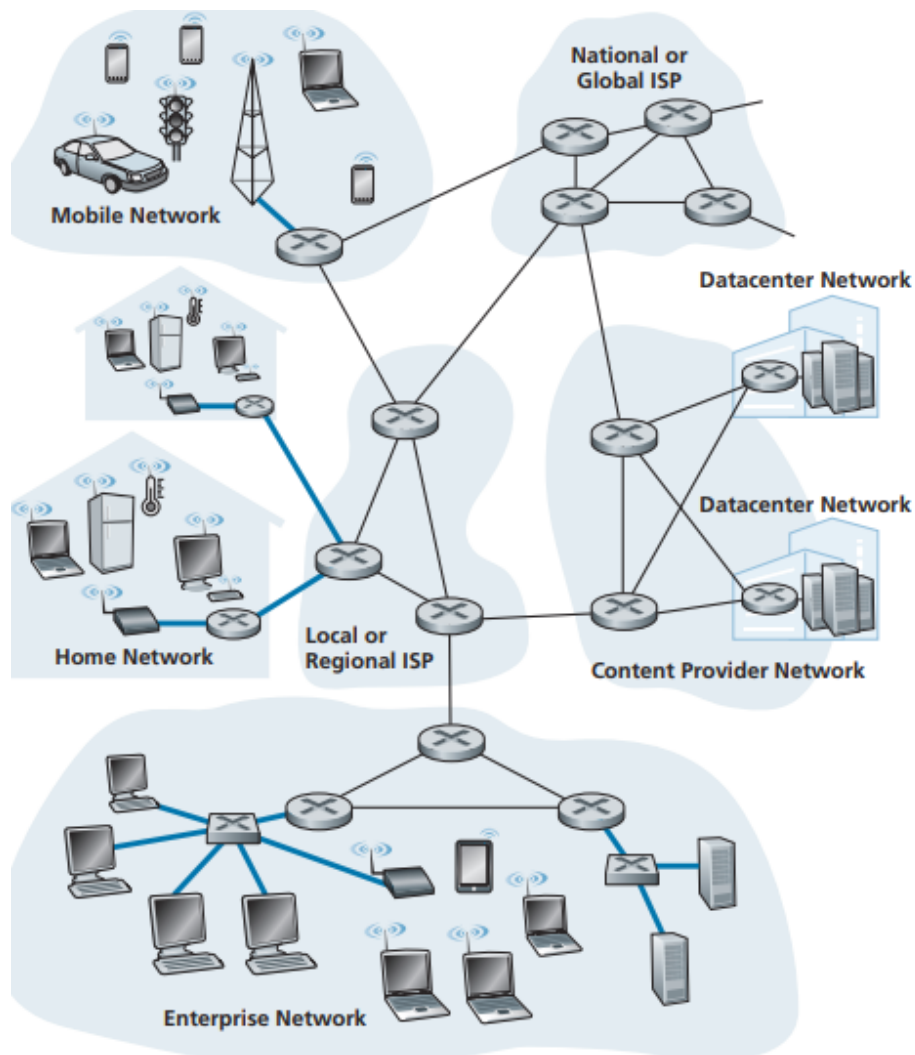


Network edge

Podemos referirnos a ellos como **host** y estos se dividirían en dos categorías **Cliente** y **Servidores**

- Cliente: suelen ser maquinas comunes, computadores, celulares, etc.
- Servidores: Suelen ser maquinas potentes o data centers.

Acceso a la red Para conectarnos a la red y usamos usualmente un router el cual se conecta con los ISPs locales, estos "routers de bordes" nos permiten conectarnos con el exterior y saltar entre islas de redes



Acceso desde casa

Hay dos maneras de conectarse al Internet desde casa que predominan, que son DSL(Línea digital de suscripción) y cable. **DSL** Los DSL usan los cables de teléfonos preexistentes, estos se conectan mediante un modem, y llegan a la compañía de telecomunicaciones y mediante un multiplexor(DSLAM) que transforma los tonos de alta frecuencia en formato digital, entonces la compañía actúa también como un ISP

- A high-speed downstream channel, in the 50 kHz to 1 MHz band
- A medium-speed upstream channel, in the 4 kHz to 50 kHz band
- An ordinary two-way telephone channel, in the 0 to 4 kHz band

This approach makes the single DSL link appear as if there were three separate links, so that a telephone call and an Internet connection can share the DSL link at

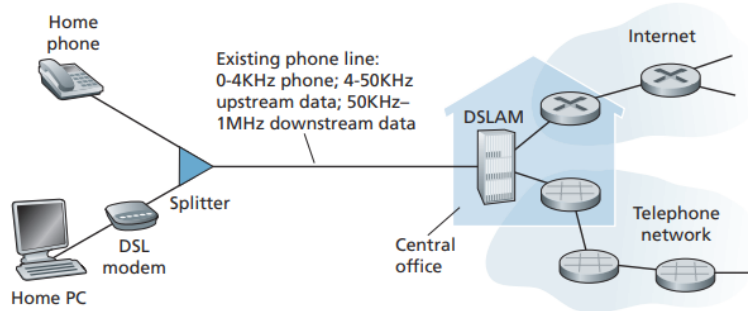


Figure 1.5 ♦ DSL Internet access

Este

sistema tiene una desventajas:

- Conexión asimétrica (diferencia entre subida y bajada)
- Peor rendimiento mientras mas lejos este físicamente el módem de la oficina central
- **Cable** Este sistema usa los cables de televisión preexistentes, cada cable soporta entre 500 y 5000 casas conectadas, usando un sistema de cable coaxial para alcanzar cada hogar esto a veces se llama HFC (fibra híbrida coaxial) . En la casa para conectarse se usa un módem de cable que va conectado por el puerto ethernet del computador el cual lleva finalmente a un CMTS similar al DSLAM en función, este tipo de conexión tiene una característica, los datos se envían por un cable compartido por lo cual la red se puede saturar, también se necesita coordinar la transmisión para evitar colisiones.
- Es asimétrico
- se puede sobrecargar el cable coaxial
- Puede haber colisión debido a que múltiples casas usan la misma red para

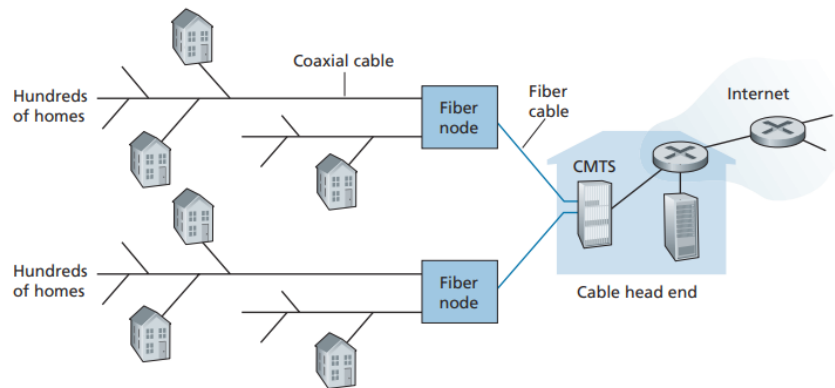
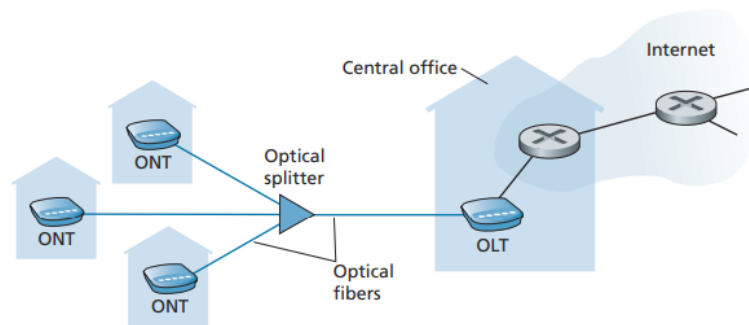


Figure 1.6 ♦ A hybrid fiber-coaxial access network

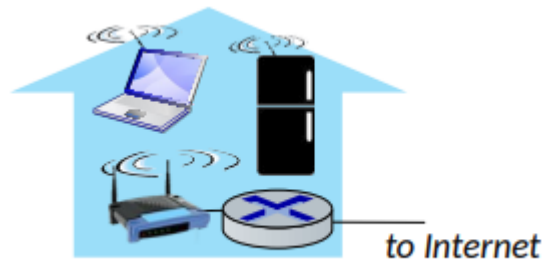
conectarse

Fibra a casa (FTTH) El concepto es tener un cable de fibra óptica que sale de la oficina central y va hacia las casas. En el sistema PON, cada casa tiene un terminador óptico de red que se conecta a un splitter del vecindario, habitualmente menos de 100 casas, luego va a un OLT que realiza la transformación de señales.



Acceso a la network mediante wireless conectamos el end system al router al cual llamaremos punto de acceso, los separamos en dos

1. **Wireless local area networks(WLANs):** Tipicamente alrededor de edificios, 802.11b/g/(WiFi): 11, 54, 450 mbps de transmisión



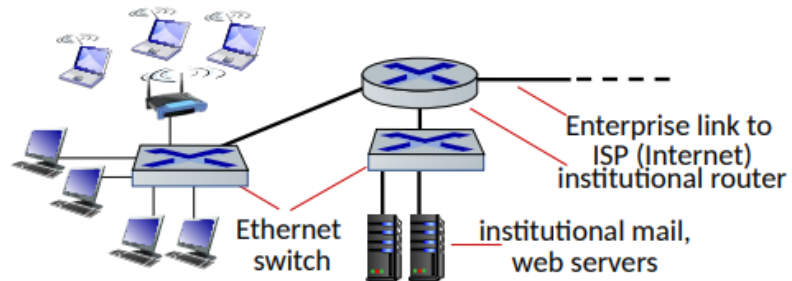
2. **Wide-area cellular access networks:** Lo provee los operadores móviles



, 10Mbps, 4G y 5G

Red de empresas

- Compañías, universidades, etc.
- Mix de cables, wireless, switches y routers



- Ethernet y wifi

Data center red Ancho de banda alto (10s to 100s gbps) conectando cientos o miles de servidores a si mismos y al Internet

Host: envíos de paquetes de data

host tiene la función de enviar:

- toman mensajes de la aplicación
- rompe en pequeños en pequeños chunks, conoce como paquetes de un largo de L bits
- transmite paquetes con un ratio de transmisión R

- transmisión de conexión, capacidad o ancho de banda

$$PacketTransmissionDelay = tiempoNecesarioParaTransmitirLBitsEnElEnlace = \frac{L(bits)}{R(bits/sec)}$$

Enlace físicos

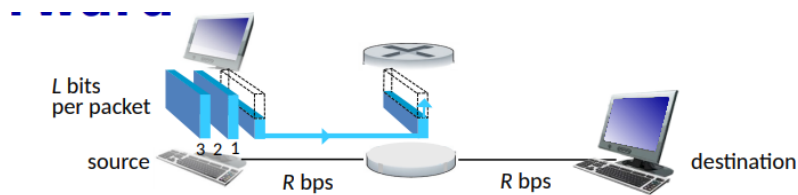
- bit: propagado entre transmisor y receptor par
- Enlace físico: lo que hay entre el transmisor y receptor
- Medio guiado: señal propagada en un medio solido por ejemplo un cable
- Medio no guiado: señal propagada libremente, ej Radio
- Twisted pair(TP): dos cables de cobre entrelazados
 - categoría 5: 100 Mbps, 1 Gbps ethernet
 - categoría 6: 10 Gbps ethernet
- Coaxial cable: dos conductores de cables concentricos, bidireccional, tiene múltiple frecuencia de los canales del cable y con unos 100 Mbps por canal
- Cables de fibra óptica: cable de cristal que llevan pulsos de luz, cada pulso es un bit, alta velocidad, bajo error ya que hay repetidores y es inmune a electromagnetismo
- Ondas de radio inalámbrico: varias bandas del espectro electromagnético, no hay un cable físico, emisor "half-duplex", es afectado por el entorno
 - Wireless LAN(WiFi)
 - Wide-area(4G)
 - Bluetooth
 - Ondas de microondas
 - satélite

Network core

- malla de routers intercomunicados
- packet-switching: el servidor rompe los mensajes de las aplicaciones en paquetes
 - la red envía paquetes de un router al otro a través de enlaces hasta su destino **forwarding**
- aka switching
- acción local: mover los paquetes entrantes al enlace correcto para seguir su camino **Routing**
- Acción global: determinar la ruta que tomara el paquete para llegar a su destino.
- algoritmo de ruta.

store and forward

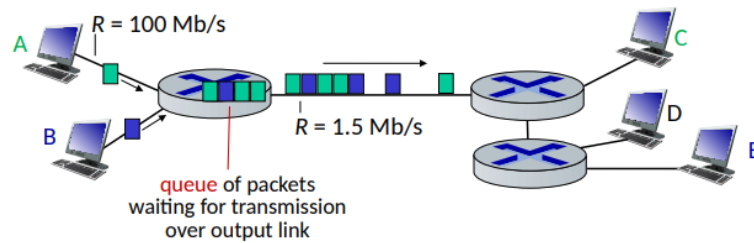
- tiene delay
- paquetes enteros llegan al router y luego son transmitidos al siguiente en-



lace.

cola

La cola llega cuando el trabajo llega mas rápido de lo que puede ser trasladado.



Los paquetes se almacenaran en la cola pero si se sobrepasa la capacidad de buffer este generara perdida de paquetes

circuit switching

Es otra forma de transmitir datos, en donde se reserva un camino entre el origen y el destino para transmitir todo los datos, garantiza el rendimiento

Circuit switching: FDM and TPM

Frecuency Division Multiplexing Dividimos la frecuencia depende la cantidad de usuarios, si tenemos 4 usuarios, a cada uno les corresponde el 25% del ancho de banda

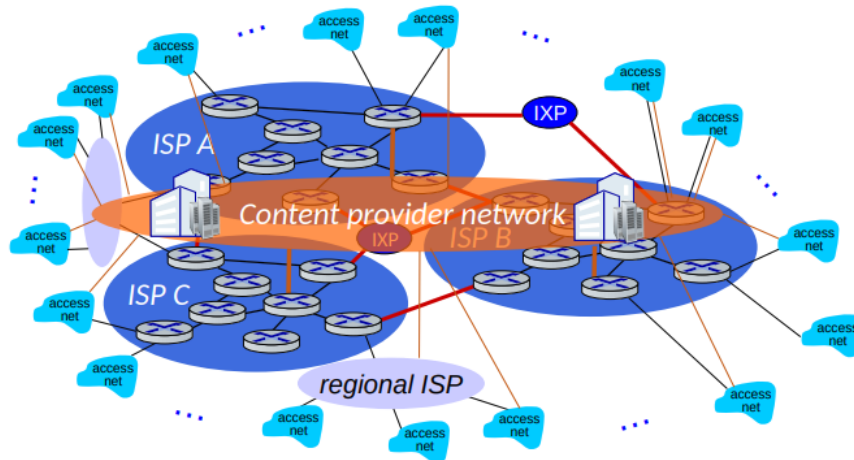
Time division Multiplexing Usamos el tiempo para dividir a los usuarios, los usuarios cuenta con el 100% de la red por un tiempo limitado, si fueran 10 segundos y 4 usuarios, serian 2,5 segundos por usuario

Packet switching vs circuit switching

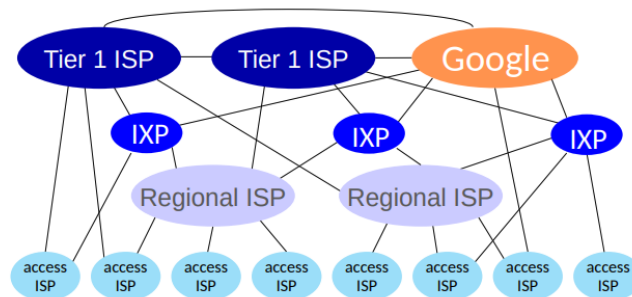
- circuit switching: 10 usuarios activos
- Packet switching: podemos tener 35 usuarios activos, donde tenemos una probabilidad de 10 activos, este es mas eficiente.
 - Se consume en base a si el usuario esta activo, consume solo cuando esta en uso

- hay una posible congestión, lo que provocaría pérdida de datos y retraso en los paquetes

Estructura de la red de redes



tenemos que dividir la red para no colapsarla, usar proveedores locales que se conectan a proveedores mas grandes



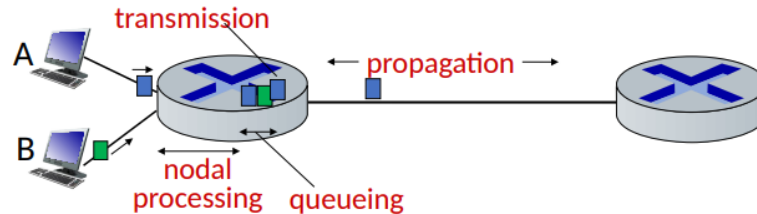
At "center": small # of well-connected large networks

- **"tier-1" commercial ISPs** (e.g., Level 3, Sprint, AT&T, NTT), national & international coverage
- **content provider networks** (e.g., Google, Facebook): private network that connects its data centers to Internet, often bypassing tier-1, regional ISPs

Performance

Perdida de paquetes

los routers tienen buffers, cuando este buffer se llena esto genera perdida de los pa-



$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

d_{proc} : nodal processing

- check bit errors
- determine output link
- typically < microsecs

d_{queue} : queueing delay

- time waiting at output link for transmission
- depends on congestion level of router

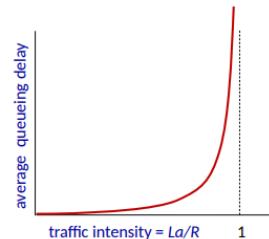
quetes

esta es la formula para calcular el delay del nodo, el delay de transmisión es distinto al de propagación

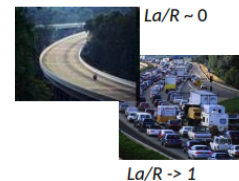
Delay de paquetes en cola

- a : average packet arrival rate
- L : packet length (bits)
- R : link bandwidth (bit transmission rate)

$$\frac{L \cdot a}{R} : \begin{array}{l} \text{arrival rate of bits} \\ \text{service rate of bits} \end{array} \quad \text{"traffic intensity"}$$



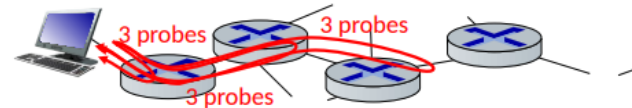
- $La/R \sim 0$: avg. queueing delay small
- $La/R \rightarrow 1$: avg. queueing delay large
- $La/R > 1$: more "work" arriving is more than can be serviced - average delay infinite!



es una manera de calcular el delay de la cola, si es cercano a 0, el delay es pequeño, si se acerca a 1 el delay es alto y si es mayor a 1 es infinito

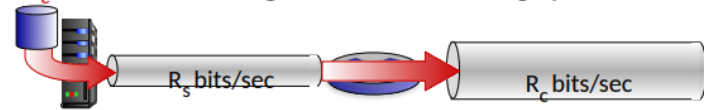
Comando traceroute

nos permite ver el delay, envía 3 paquetes al destino

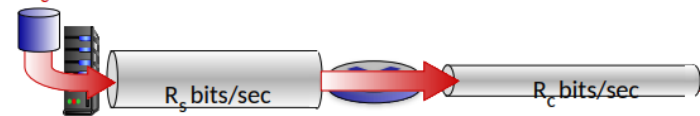


Throughput

$R_s < R_c$ What is average end-end throughput?



$R_s > R_c$ What is average end-end throughput?



bottleneck link

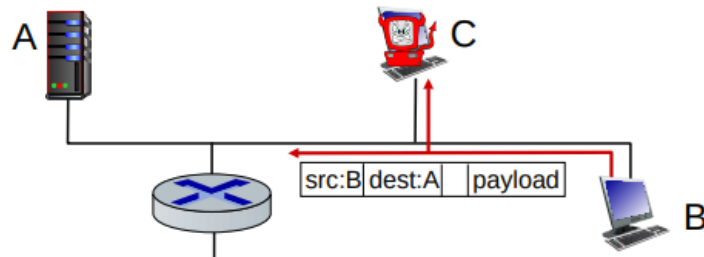
link on end-end path that constrains end-end throughput

es el ratio entre el que envía y el que recibe
aquí se genera un cuello de botella debido a que el más pequeño pone el límite

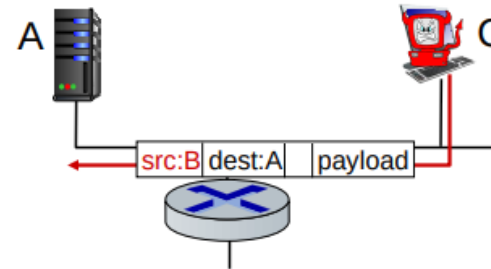
Seguridad

Sniffing de paquetes

la tarjeta en modo promiscuo puede leer todos los paquetes, escucha todo

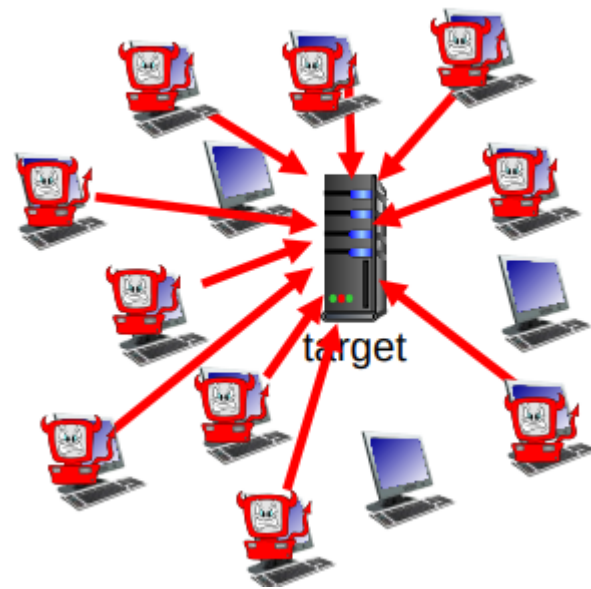


Falsa identidad



IP spoofing: inyecta un paquete con un origen de direccion falso

Denegación de servicios



sobrecargan el objetivo, mediante peticiones mal formadas

lineas de defensas

- autenticacion
- confidencialidad
- chequeo de integridad
- restriccion de acceso
- firewall

Capas de protocolos y servicios

ticket (purchase)	<i>ticketing service</i>	ticket (complain)
baggage (check)	<i>baggage service</i>	baggage (claim)
gates (load)	<i>gate service</i>	gates (unload)
runway takeoff	<i>runway service</i>	runway landing
airplane routing	<i>routing service</i>	airplane routing

layers: each layer implements a service

- via its own internal-layer actions
- relying on services provided by layer below

Cada capa presenta un servicio

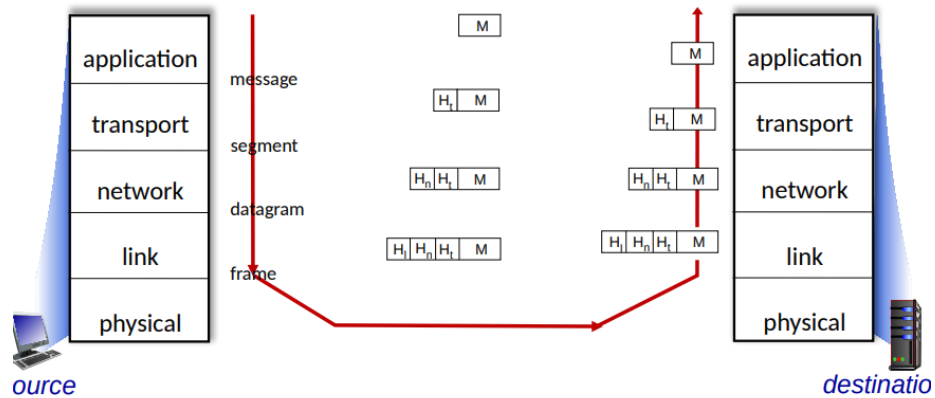
Usar el modelo de capas mejora la mantención.

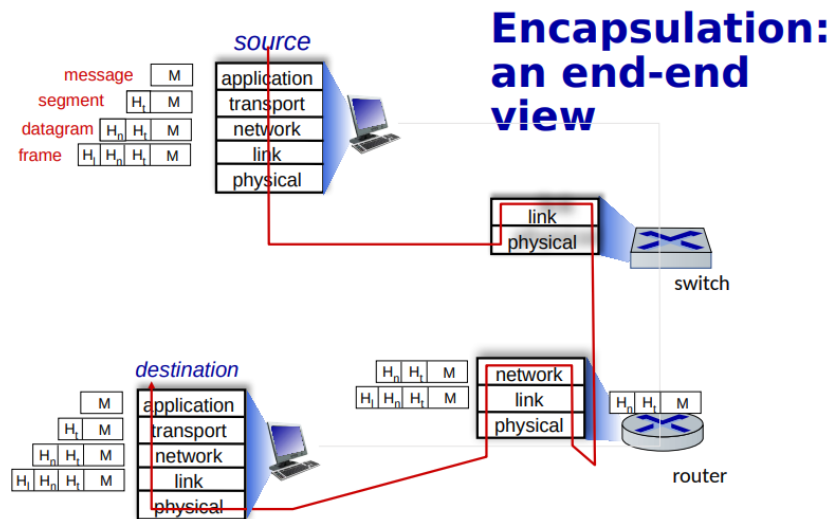
stack de las capas de los protocolos

- **application:** supporting network applications
 - HTTP, IMAP, SMTP, DNS
- **transport:** process-process data transfer
 - TCP, UDP
- **network:** routing of datagrams from source to destination
 - IP, routing protocols
- **link:** data transfer between neighboring network elements
 - Ethernet, 802.11 (WiFi), PPP
- **physical:** bits “on the wire”



Services, Layering and Encapsulation





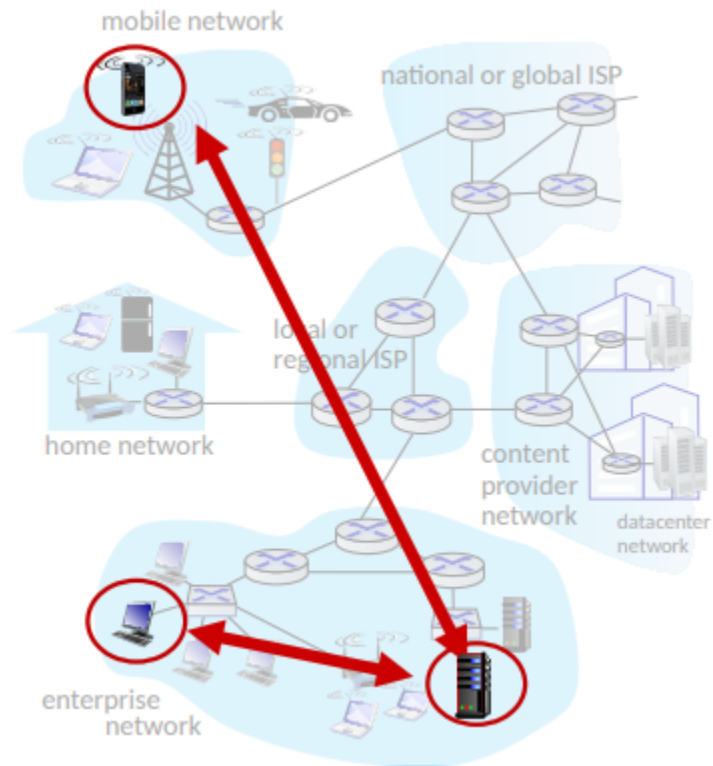
En resumen los protocolos tienen una estructura al enviarse "se arma de una manera", y al llegar al destino se "desarma" para extraer la información

ch2: capa de aplicación

Principios

paradigma cliente-servidor

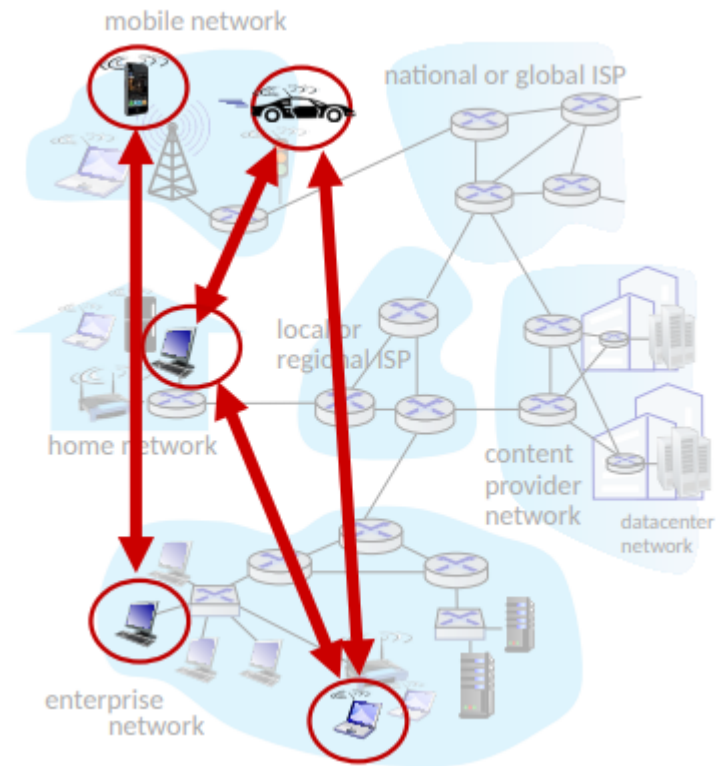
- **Host:** Ip estática, siempre hosteando y a veces en datacenters.
- **Cliente:** Ip dinámica, se contacta con el servidor, no se comunica con



otros clientes.

P2P Arquitectura

- No siempre en servidor
- Sistema arbitrario de fin del sistema
- los pares hace peticiones de un servicio al otro y otro par lo retorna



- cambia de ip

clients, servers

client process: process that initiates communication

server process: process that waits to be contacted

Socket El socket es un análogo a una puerta, es una comunicación donde se reciben y se envían mensajes, es como una comunicación entre dos extremos donde no se interactúa con el exterior, tienen que estar en el mismo socket para comunicarse.

Proceso de direcciones Para enviar algo necesitamos la ip y el puerto, esa son las direcciones en internet.

Protocolos Existen protocolos abiertos y cerrados, donde los cerrados solo los

open protocols:

- defined in RFCs, everyone has access to protocol definition
- allows for interoperability
- e.g., HTTP, SMTP

proprietary protocols:

- e.g., Skype, Zoom

propietarios saben como funciona

Que necesitan los servicios?

- Necesitan integridad de datos(algunos aceptan perdida)
- Timing, necesitamos un delay bajo
- throughput, muchas apps necesitan poco, bits/s

application	data loss	throughput	time sensitive?
file transfer/download	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5Kbps-1Mbps video:10Kbps-5Mbps	yes, 10's msec
streaming audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

- Seguridad, encriptacion de datos

Protocolos de servicio

1. TCP

- Garantiza la integridad de los datos
- El emisor no puede saturar al receptor
- tiene control de congestión
- se requiere un proceso entre el cliente y el receptor
- no provee: timing, throughput ni seguridad

2. UDP

- No garantiza la integridad de datos
- no provee. integridad, control de flujo, control de congestión, timing,

TCP service:

- **reliable transport** between sending and receiving process
- **flow control**: sender won't overwhelm receiver
- **congestion control**: throttle sender when network overloaded
- **connection-oriented**: setup required between client and server processes
- **does not provide**: timing, minimum throughput guarantee, security

UDP service:

- **unreliable data transfer** between sending and receiving process
- **does not provide**: reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup.

Q: why bother? Why is there a UDP?

throughput, seguridad o conexión.

application	application layer protocol	transport protocol
file transfer/download	FTP [RFC 959]	TCP
e-mail	SMTP [RFC 5321]	TCP
Web documents	HTTP 1.1 [RFC 7320]	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary	TCP or UDP
streaming audio/video	HTTP [RFC 7320], DASH	TCP
interactive games	WOW, FPS (proprietary)	UDP or TCP

Seguridad TCP

1. Vanilla TCP y UDP

- Esta todo en texto plano
- No es seguro

2. Transport Layer Security(TLS)

- Esta encriptado
- Integridad de datos
- Autenticación por endpoint

Web y HTTP

Para acceder a una web primero ponemos el nombre del host y luego el path

www.someschool.edu / someDept/pic.gif

host name

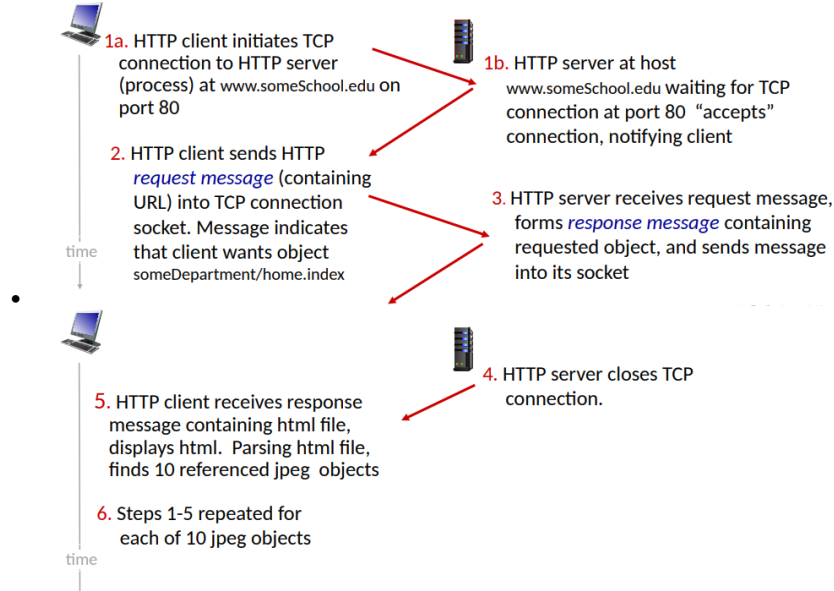
path name

Vista de HTTP http(hypertext transfer protocol) es un protocolo, que el cliente recibe y permite al navegador desplegar la pagina y el servidor envía este protocolo. http usa TCP, o sea el cliente manda una petición por el puerto 80, es recibido y se cierra la conexión, este protocolo no guarda la información del cliente anterior.

Conexiones HTTP: dos tipos:

1. No persistente HTTP

- Se abre la conexión
- se envía el objeto
- se cierra la conexión



2. HTTP persistente

- se abre la conexión TCP
- Muchos objetos se envían por este socket
- se cierra la conexión

HTTP no persistente

Palabra	Definición
RTT	Tiempo en que un paquete pequeño viaja del cliente al servidor y de vuelta

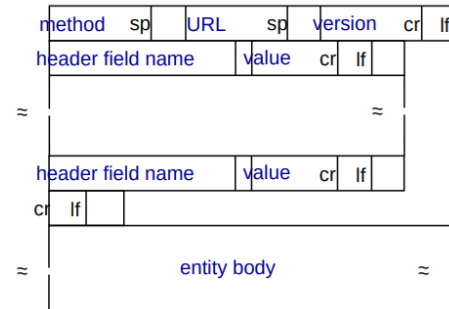
HTTP response time(per object):

- Un RTT inicia la conexión TCP
- Se crea una respuesta
- Object/File transmisión

$$NonPersistentHTTPResponseTime = 2RTT + FileTransmissionTime$$

Las conexiones no persistentes tienen unos problemas, requiere dos RTTs por objeto, el sistema crea overhead por cada conexión TCP y los navegadores ofrecen múltiples conexiones TCP de manera paralela, para referenciar objetos

HTTP persistente (HTTP1.1) Se mantiene la conexión abierta por lo que no requerimos de muchas llamadas RTT para cargar los objetos, si no que se abre la conexión y se mantiene abierta para el envío de objetos, logrando reducir el tiempo de carga



Request mensajes se hace en ASCII. Formato general

POST method:

- web page often includes form input
- user input sent from client to server in entity body of HTTP POST request message

HEAD method:

- requests headers (only) that would be returned if specified URL were requested with an HTTP GET method.

GET method (for sending data to server):

- include user data in URL field of HTTP GET request message (following a '?'):
www.somesite.com/animalsearch?monkeys&banana

PUT method:

- uploads new file (object) to server
- completely replaces file that exists at specified URL with content in entity body of POST HTTP request message

Estas son peticiones request:

200 OK

- request succeeded, requested object later in this message

301 Moved Permanently

- requested object moved, new location specified later in this message (in Location: field)

400 Bad Request

- request msg not understood by server

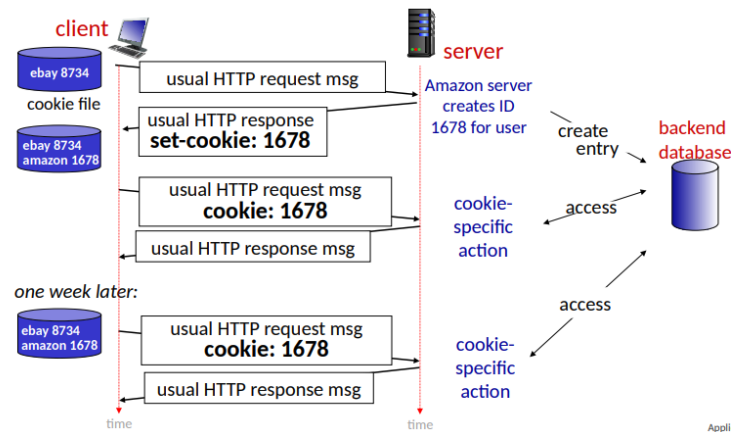
404 Not Found

- requested document not found on this server

505 HTTP Version Not Supported

y existen varios códigos de respuestas:

Cookies Muchos navegadores usan cookies para mantener el estado de una transacción, las cookies se mantienen en el host de usuario administrado por el navegador y también una parte se guarda en la base de datos del sitio web.



Application Layer: 2-34

aside

cookies and privacy:

- cookies permit sites to *learn* a lot about you on their site.
- third party persistent cookies (tracking cookies) allow common identity (cookie value) to be tracked across multiple web sites

Web Caches La web cache es una copia del html que se guarda de manera local en el navegador, este hace una consulta en el servidor para ver si la versión guardada sigue estando actualizada, se devuelve una cache al cliente (una versión guardada de manera local de la página). También se conocen como proxy servers, actúa en ambos lados (cliente/servidor), el header dice como almacenar

Cache-Control: max-age=<seconds>

Cache-Control: no-cache

la cache

Reduce el tiempo de petición, la cache esta mas cerca del cliente, reduce el trafico en los enlaces de instituciones y el Internet esta lleno de cache

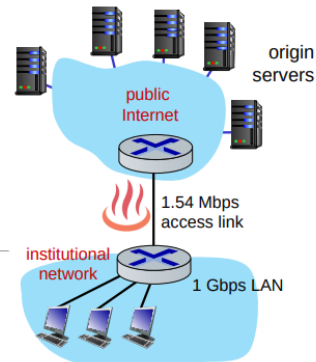
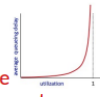
Caching example

Scenario:

- access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
 - avg data rate to browsers: 1.50 Mbps

Performance:

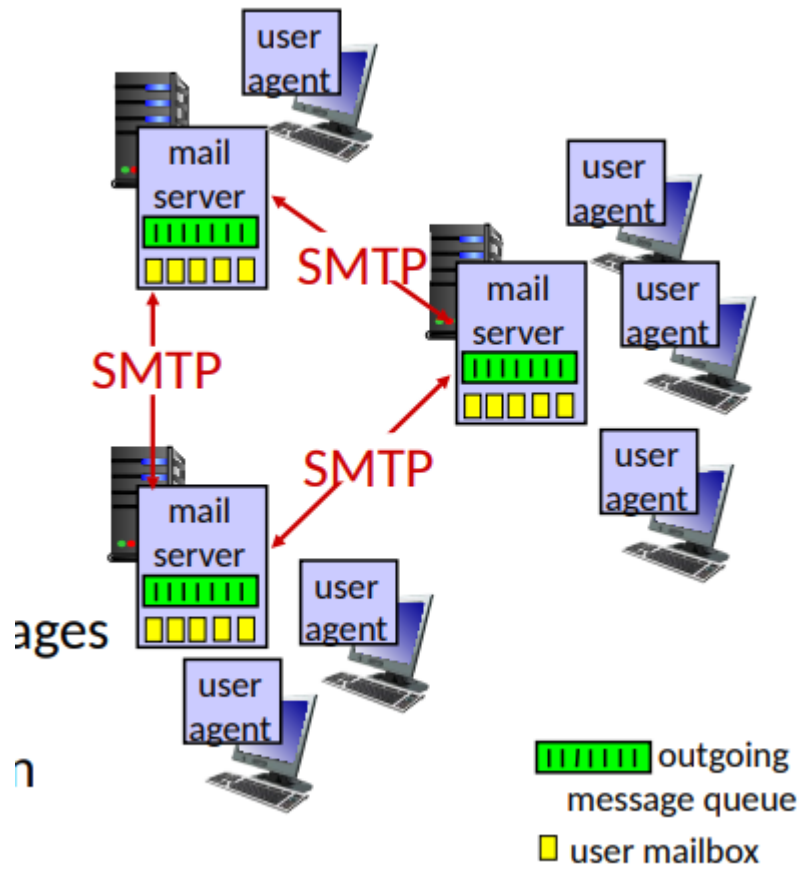
- access link utilization = .97 *problem: large queueing delays at high utilization!*
- LAN utilization: .0015
- end-end delay = Internet delay + access link delay + LAN delay
= 2 sec + minutes + usecs



Application Layer: :

En la actualidad las web caches no son del todo necesarias debido a que la mayoría de paginas son dinámicas(con backend) y que tenemos altas velocidades de red

Email, SMTP, IMAP



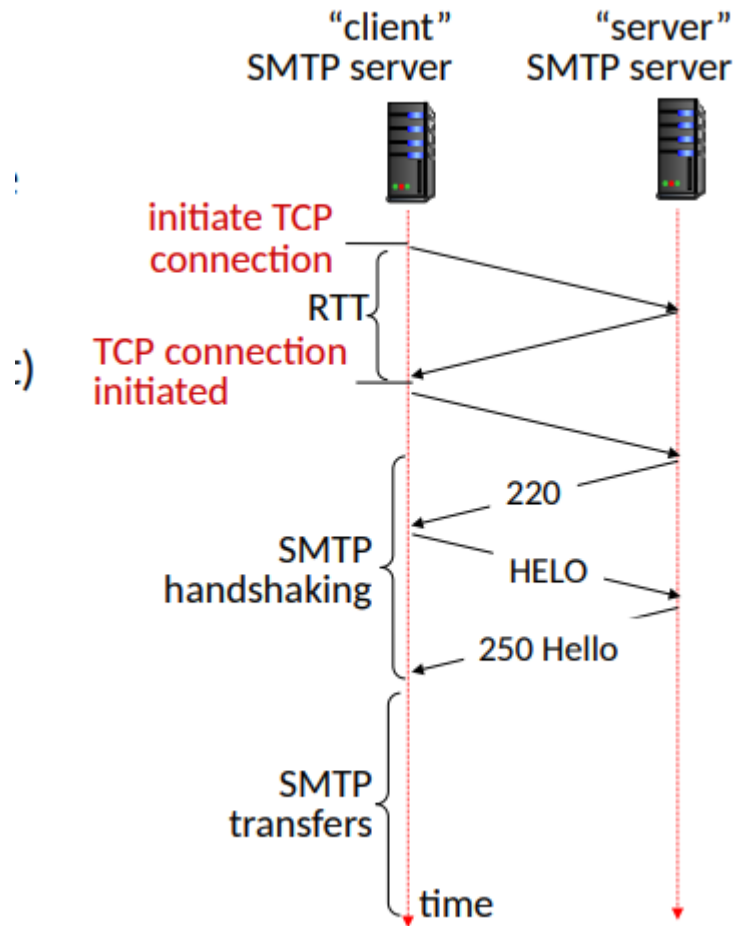
Email

Tiene 3 componentes mayores

- User agents
- Mail servers:
 - MailBox: Contiene los mensajes entrantes para el usuario
 - Message queue: cola de mensajes salientes que van a ser enviados
- Simple mail transfer protocol (SMTP):
 - Entre servidores de emails para enviar mensajes
 - * Cliente: envía al servidor de mails
 - * "Servidor": Recibiendo servidores de mails

SMTP RFC(5321) usa TCP para garantizar la correcta transacción de información entre datos, se usa el puerto 25

- Transferencia directa: Envía al servidor(actuando como cliente) para recibir al servidor Hay 3 fases de la transferencia:
 1. SMTP Handshaking
 2. SMTP transferencia de mensajes
 3. SMTP cierre Los comandos son en ASCII y las respuestas son con códigos y



frases como en HTTP

comparison with HTTP:

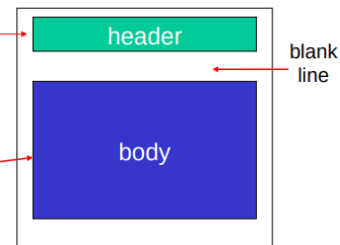
- HTTP: client pull
- SMTP: client push
- both have ASCII command/response interaction, status codes
- HTTP: each object encapsulated in its own response message
- SMTP: multiple objects sent in multipart message
- SMTP uses persistent connections
- SMTP requires message (header & body) to be in 7-bit ASCII
- SMTP server uses CRLF.CRLF to determine end of message

Mail message format

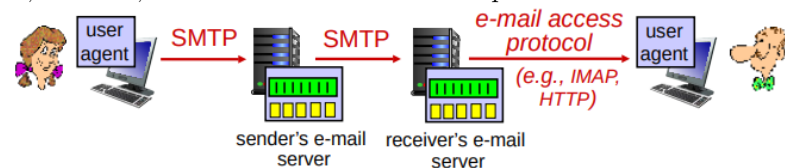
SMTP: protocol for exchanging e-mail messages, defined in RFC 5321 (like RFC 7231 defines HTTP)

RFC 2822 defines *syntax* for e-mail message itself (like HTML defines syntax for web documents)

- header lines, e.g.,
 - To:
 - From:
 - Subject:these lines, within the body of the email message area different from SMTP MAIL FROM:, RCPT TO: commands!
- Body: the "message", ASCII characters only



Recuperando mails El SMTP almacena y envía mensajes, es un servidor. Luego tenemos IMAP(Internet Mail Access Protocol) que nos permite acceder vía web a nuestros correos almacenados, podemos eliminar, almacenar,etc. Gmail, Outlook, Hotmail, etc nos dan una interfaz web para acceder a nuestros



correos.

DNS: Domain Name System

Nos permite acceder mas fácil a las paginas, ya que crea un enlace entre un nombre y una ip, de esta manera accedemos a usm.cl y no a 213.133.12.1. Es una base de datos distribuida que esta implementada en las librerias de muchos servidores. Esta en la capa de aplicación: host, DNS resuelve nombres.

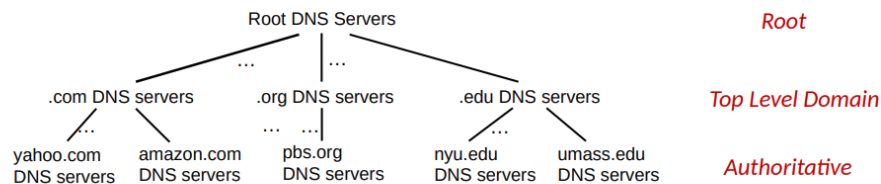
- Es el núcleo del Internet
- es complejo en los límites de la red

DNS: Servicios y estructura

- Hostnames a direcciones ip.
- Alias de host.
- Alias de servidores de mail.
- Distribución de carga. **No podemos tener un DNS centralizado debido a que si falla se cae internet, tiene un volumen de tráfico alto, una base de datos distante y el mantenimiento** Si pensamos en los DNS:
- Es una base de datos distribuidas y homogénea
- Maneja trillones de consultas
- Organizado pero físicamente descentralizado
- Aprueba de balas

DNS: distribuido y jerárquico

DNS: a distributed, hierarchical database



Client wants IP address for www.amazon.com; 1st approximation:

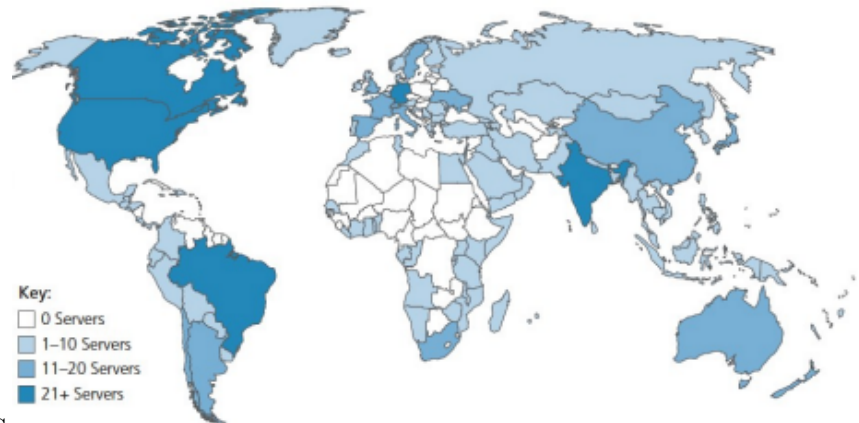
- client queries root server to find .com DNS server
- client queries .com DNS server to get amazon.com DNS server
- client queries amazon.com DNS server to get IP address for www.amazon.com

DNS: root name servers

Lo ideal es que a nivel top y autoritativo pueda resolver todas mis consultas, la última opción es consultar al root.

- El internet no funciona sin el
- DNSSEC provee seguridad (autenticación e integridad de mensajes)
- ICANN (Internet Corporation for assigned Names and Numbers) Adminis-

13 logical root name “servers”
worldwide each “server” replicated
many times (~200 servers in US)



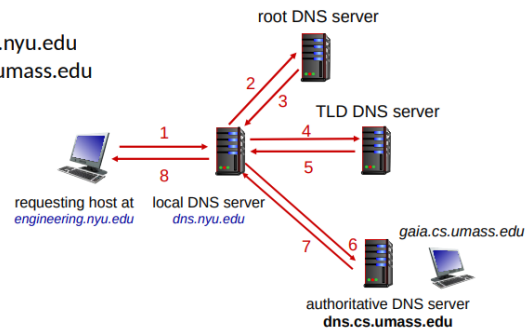
tra los root DNS

DNS name resolution: iterated query

Example: host at engineering.nyu.edu
wants IP address for gaia.cs.umass.edu

Iterated query:

- contacted server replies with name of server to contact
- “I don’t know this name, but ask this server”



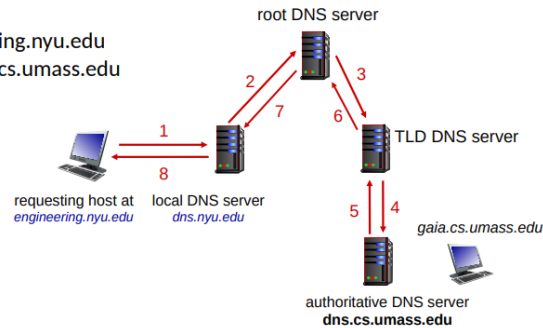
Los DNS locales suelen tener cache para acelerar la operación

DNS name resolution: recursive query

Example: host at engineering.nyu.edu
wants IP address for gaia.cs.umass.edu

Recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?



Caching DNS Information

- once (any) name server learns mapping, it *caches* mapping, and *immediately* returns a cached mapping in response to a query
 - caching improves response time
 - cache entries timeout (disappear) after some time (TTL)
 - TLD servers typically cached in local name servers
- cached entries may be *out-of-date*
 - if named host changes IP address, may not be known Internet-wide until all TTLs expire!
 - *best-effort name-to-address translation!*

DNS records

DNS: distributed database storing resource records (RR)

RR format: (name, value, type, ttl)

type=A

- name is hostname
- value is IP address

type=NS

- name is domain (e.g., foo.com)
- value is hostname of authoritative name server for this domain

type=CNAME

- name is alias name for some "canonical" (the real) name
- www.ibm.com is really servereast.backup2.ibm.com
- value is canonical name

type=MX

- value is name of SMTP mail server associated with name

DNS protocol messages

DNS *query* and *reply* messages, both have same *format*:

message header:

- **identification**: 16 bit # for query, reply to query uses same #

▪ **flags**:

- query or reply
- recursion desired
- recursion available
- reply is authoritative

← 2-bytes → ← 2-bytes →	
identification	flags
# questions	# answer RRs
# authority RRs	# additional RRs
questions (variable # of questions)	
answers (variable # of RRs)	
authority (variable # of RRs)	
additional info (variable # of RRs)	